Manual-aula do código — Cuidados à Mente

Versão 2026-10-12. Cada aula liga ação na tela, regra no servidor e dado persistido. As linhas copiadas vêm do código atual; gere novamente este manual quando alterar a fonte.



Do aplicativo à operação

Use a imagem para entender o caminho. Depois confira as linhas da fonte e a explicação de cada funcionalidade.

1. Entenda a arquitetura

A tela chama requisitarApi. O cliente envia token e JSON. O backend identifica a conta, verifica aceite e permissão para a pessoa, valida campos e consulta SQLite. Novas mídias ficam em pasta privada; módulos Expo SDK 57 cuidam de notificações, captura pontual e armazenamento nativo.



Arquitetura geral do cuidado

```
Tela React Native -> cliente-api -> controlador Java
                                 -> BancoDados -> SQLite
                                 -> MidiaController -> arquivos privados
Aparelho -> SecureStore / SQLCipher / Notifications / Location
```

Leia uma aula de cada vez e reproduza com dados de teste. Um trecho isolado não substitui as verificações do método completo.

2. Cadastro mínimo

Na interface: Tela inicial e Contas de idosos.



Figura 1 — Cadastro mínimo

O que a funcionalidade faz: CPF, RG, nascimento, contato e endereço são opcionais.

Fonte: `backend/src/main/java/com/cuidadosamente/backend/controlador/CuidadosController.java`, linha 863. Os números à esquerda são as linhas reais.

```
863 |     String representacao = campoObrigatorio(dados, "representacao", 60);
864 |     if (!Set.of("titular_autorizou", "representante_legal").contains(representacao))
{
865 |       throw erroDadosInvalidos("Informe quem autorizou o cadastro.");
866 |     }
867 |     banco.executarTransacao(
868 |       new Object[] {
869 |         "INSERT INTO idosos(id,owner_id,nome,nascimento,observacoes,perfil) VALUES(?,
?,?,?,?,?)",
870 |         id,
871 |         contaAutorizadaId,
872 |         campoObrigatorio(dados, "nome", 150),
873 |         nascimento,
874 |         campoOpcional(dados, "observacoes", 2000),
875 |         conversorJson.writeValueAsString(perfilIdoso(dados, Map.of())),
876 |       },
877 |       new Object[] {
878 |         "INSERT INTO subject_consents(idoso_id,actor_id,representation,accepted_at)
VALUES(?,?,?,?)",
879 |         id,
880 |         contaAutorizadaId,
881 |         representacao,
882 |         Instant.now().toString(),
883 |       },
884 |       evento(
```

A validação exige autorização e um dos papéis aceitos. A transação grava cadastro, declaração e evento juntos.

Experimente: No cadastro da pessoa, informe nome e confirme autorização válida.

Limites: O perfil Responsável não comprova representação legal.

3. Acesso por QR

Na interface: Contas de idosos e botão QR da tela inicial.



Figura 2 — Acesso por QR

O que a funcionalidade faz: A conta idosa acessa somente a pessoa vinculada.

Fonte: `backend/src/main/java/com/cuidadosamente/backend/controlador/CuidadosController.java`, linha 432. Os números à esquerda são as linhas reais.

```
432 |     byte[] bytes = new byte[32];
433 |     new SecureRandom().nextBytes(bytes);
434 |     String code = Base64.getUrlEncoder().withoutPadding().encodeToString(bytes);
435 |     long expires = Instant.now().plusSeconds(600).getEpochSecond();
436 |     banco.executar(
437 |       "INSERT INTO qr_access(token_hash,idoso_id,expires) VALUES(?,?,?) ON " +
438 |         "CONFLICT(idoso_id) DO UPDATE SET " +
439 |         "token_hash=excluded.token_hash,expires=excluded.expires",
440 |       gerarHash(code),
441 |       id,
442 |       expires
443 |     );
444 |     return Map.of(
445 |       "codigo",
446 |       code,
```

São gerados 32 bytes aleatórios. Apenas o hash fica no banco; a sessão ganha identificador e validade de doze horas.

Experimente: Confira o nome e aceite os documentos quando solicitado.

Limites: QR vale dez minutos e é de uso único. Desconectar encerra sessões, desativa a senha opcional e preserva registros.

4. Rotinas, medicamentos e consultas

Na interface: Painel e abas dos registros.



Figura 3 — Rotinas, medicamentos e consultas

O que a funcionalidade faz: Pesquisa e resumos usam apenas registros autorizados. Concluir registra a declaração do usuário.

Fonte: `backend/src/main/java/com/cuidadosamente/backend/controlador/CuidadosController.java`, linha 1240. Os números à esquerda são as linhas reais.

```
1240 |     String mutation = campoOpcional(dados, "mutationId", 80);
1241 |     if (!mutation.isEmpty()) {
1242 |       var receipt = banco.consultar(
1243 |         "SELECT record_id FROM mutation_receipts WHERE id=? AND user_id=?",
1244 |         mutation,
1245 |         contaAutorizadaId
1246 |       );
1247 |       if (!receipt.isEmpty()) {
1248 |         var anterior = buscarRegistro(
1249 |           receipt.getFirst().get("record_id").toString(),
1250 |           contaAutorizadaId
1251 |         );
1252 |         if (!idoso.equals(anterior.get("idosoId")) || !tipo.equals(anterior.
get("tipo"))) {
1253 |           throw new ResponseStatusException(
1254 |             HttpStatus.CONFLICT,
1255 |             "Esta tentativa pertence a outro cadastro ou tipo de registro.
Recarregue " +
1256 |               "antes de tentar novamente."
1257 |           );
1258 |         }
1259 |         return anterior;
```

Primeiro são validados tipo e permissão. mutationId reconhece criação reenviada para evitar duplicação.

Experimente: Use Concluir ou Reabrir conforme a permissão.

Limites: O aplicativo não mede ingestão, prescreve doses nem agenda consultas com estabelecimentos.

5. Histórico de autoria

Na interface: Acompanhamento > Histórico.



Figura 4 — Histórico de autoria

O que a funcionalidade faz: Novas mudanças geram eventos separados, na mesma transação do registro.

Fonte: `backend/src/main/java/com/cuidadosamente/backend/controlador/CuidadosController.java`, linha 2018. Os números à esquerda são as linhas reais.

```
2018 |     var estadoAnterior = new LinkedHashMap<>(antes);
2019 |     var estadoPosterior = new LinkedHashMap<>(depois);
2020 |     estadoAnterior.remove("midia");
2021 |     estadoPosterior.remove("midia");
2022 |     return new Object[] {
2023 |       "INSERT INTO audit_events(id,idoso_id,actor_id,actor_name,tipo,entity_id,
action,occurred_at,before_json,after_json) VALUES(?,?,?,?,?,?,?,?,?,?)",
2024 |       UUID.randomUUID().toString(),
2025 |       pessoa,
2026 |       autor,
2027 |       buscarUsuario(autor).get("nome"),
2028 |       tipo,
2029 |       entidade,
2030 |       acao,
2031 |       Instant.now().toString(),
2032 |       conversorJson.writeValueAsString(estadoAnterior),
2033 |       conversorJson.writeValueAsString(estadoPosterior),
2034 |     };
2035 |   }
2036 |
2037 |   Map<String, Object> contatoEmergencia(String pessoa) {
2038 |     var contatos = banco.consultar(
2039 |       "SELECT nome,telefone,nome_alternativo AS " +
```

O servidor usa o autor autenticado, copia os estados, retira mídia e grava UUID e horário UTC na trilha.

Experimente: Compare Antes e Depois para entender uma correção.

Limites: Não reconstrói autoria antiga. Exibe até 300 eventos; retenção padrão de 365 dias.

6. Diário e passagem de turno

Na interface: Acompanhamento > Diário.



Figura 5 — Diário e passagem de turno

O que a funcionalidade faz: As entradas anteriores são preservadas. Adultos usam permissões de rotina.

Fonte: `backend/src/main/java/com/cuidadosamente/backend/controlador/EvolucaoController.java`, linha 612. Os números à esquerda são as linhas reais.

```
612 |   public synchronized Map<String, Object> anotar(
613 |     @RequestHeader(value = "Authorization", required = false) String autorizacao,
614 |     @PathVariable String id,
615 |     @RequestBody Map<String, Object> dados
616 |   ) {
617 |     String usuarioId = cuidados.exigirAdulto(autorizacao);
618 |     cuidados.exigirPermissao(id, usuarioId, "criar_rotinas");
619 |     String kind = cuidados.campoObrigatorio(dados, "categoria", 30);
620 |     String texto = cuidados.campoObrigatorio(dados, "texto", 4000);
621 |     String nid = UUID.randomUUID().toString();
622 |     if (!Set.of("observacao", "passagem_turno", "correcao").contains(kind)) {
623 |       throw cuidados.erroDadosInvalidos("Categoria inválida.");
624 |     }
625 |     banco.executarTransacao(
626 |       new Object[] {
627 |         "INSERT INTO care_notes(id,idoso_id,actor_id,actor_name,kind,texto,
occurred_at) VALUES(?,?,?,?,?,?,?)",
628 |         nid,
629 |         id,
630 |         usuarioId,
631 |         cuidados.buscarUsuario(usuarioId).get("nome"),
632 |         kind,
```

Exige adulto e criar_rotinas. Aceita três categorias e grava autor, texto e horário com evento de auditoria.

Experimente: Salve e confira autor e horário. Para corrigir, acrescente outra entrada.

Limites: Exibe até 200 entradas. Não substitui documentação clínica formal.

7. Convites e retirada de vínculo

Na interface: Perfil > Permissões e Configurações.



Figura 6 — Convites e retirada de vínculo

O que a funcionalidade faz: Antes do aceite, os registros continuam inacessíveis. A retirada elimina os acessos do vínculo.

Fonte: `backend/src/main/java/com/cuidadosamente/backend/controlador/EvolucaoController.java`, linha 263. Os números à esquerda são as linhas reais.

```
263 |       var c = linhas.getFirst();
264 |       String pessoa = c.get("idoso_id").toString();
265 |       if (!(dados.get("aceitar") instanceof Boolean aceitar)) {
266 |         throw cuidados.erroDadosInvalidos("Responda ao convite.");
267 |       }
268 |       if (aceitar) {
269 |         if (!cuidados.donoDosDados(pessoa).equals(c.get("invited_by"))) {
270 |           throw new ResponseStatusException(
271 |             HttpStatus.CONFLICT,
272 |             "A administração mudou. Peça um novo convite."
273 |           );
274 |         }
275 |         var comandos = new ArrayList<Object[]>();
276 |         comandos.add(new Object[] {
277 |           "INSERT INTO caregiver_links(idoso_id,user_id,expires_at,reviewed_at)
VALUES(?,?,?,?)",
278 |           pessoa,
279 |           usuarioId,
280 |           LocalDate.now().plusDays(90).toString(),
281 |           Instant.now().toString(),
```

A consulta anterior restringe destinatário e validade. O aceite confere o dono atual e cria vínculo e permissões em transação.

Experimente: Para sair, use Acompanhamento > Administração > Retirar meu acesso.

Limites: Convite vale sete dias. Antes do aceite aparece quem convidou, sem dados de cuidado.

8. Validade e revisão de acessos

Na interface: Acompanhamento > Administração > Revisar acessos.



Figura 7 — Validade e revisão de acessos

O que a funcionalidade faz: Vínculos aceitos começam com 90 dias. O servidor bloqueia acesso vencido.

Fonte: `backend/src/main/java/com/cuidadosamente/backend/controlador/CuidadosController.java`, linha 1537. Os números à esquerda são as linhas reais.

```
1537 |   Set<String> permissoesEfetivas(String idosoId, String usuarioId) {
1538 |     if (ehDono(idosoId, usuarioId)) {
1539 |       return PERMISSOES;
1540 |     }
1541 |     if (
1542 |       !banco
1543 |         .consultar(
1544 |           "SELECT 1 FROM elder_accounts WHERE idoso_id=? AND user_id=?",
1545 |           idosoId,
1546 |           usuarioId
1547 |         )
1548 |         .isEmpty()
1549 |     ) {
1550 |       return permissoesIdoso(idosoId);
1551 |     }
1552 |     if (
1553 |       !banco
1554 |         .consultar(
1555 |           "SELECT 1 FROM caregiver_links WHERE idoso_id=? AND user_id=? AND
(expires_at='' OR expires_at>?)",
1556 |           idosoId,
```

O método confere propriedade e vínculo válido. A consulta ilustrada bloqueia vínculo vencido; na sequência, a política determina as permissões efetivas. Consulte a função completa no arquivo indicado.

Experimente: Confirme a revisão ou retire acessos antigos.

Limites: Renovação limitada a um ano. Vínculos antigos recebem 90 dias a partir da migração.

9. Transferência de administração

Na interface: Acompanhamento > Administração e Configurações do destinatário.



Figura 8 — Transferência de administração

O que a funcionalidade faz: Preserva registros e retira acesso do administrador anterior.

Fonte: `backend/src/main/java/com/cuidadosamente/backend/controlador/EvolucaoController.java`, linha 461. Os números à esquerda são as linhas reais.

```
461 |       if (!administradorAnterior.equals(cuidados.donoDosDados(pessoa))) {
462 |         throw new ResponseStatusException(HttpStatus.CONFLICT, "A administração já
mudou.");
463 |       }
464 |       banco.executarTransacao(
465 |         new Object[] {
466 |           "UPDATE idosos SET owner_id=? WHERE id=? AND owner_id=?",
467 |           usuarioId,
468 |           pessoa,
469 |           administradorAnterior,
470 |         },
471 |         new Object[] {
472 |           "UPDATE registros SET owner_id=?,versao=versao+1 WHERE idoso_id=?",
473 |           usuarioId,
474 |           pessoa,
475 |         },
476 |         new Object[] { "DELETE FROM caregiver_invitations WHERE idoso_id=?", pessoa
},
477 |         new Object[] { "DELETE FROM access_grants WHERE idoso_id=?", pessoa },
478 |         new Object[] { "DELETE FROM access_policies WHERE idoso_id=?", pessoa },
479 |         new Object[] { "DELETE FROM caregiver_links WHERE idoso_id=?", pessoa },
480 |         new Object[] { "DELETE FROM location_access WHERE idoso_id=?", pessoa },
481 |         new Object[] { "DELETE FROM location_points WHERE idoso_id=?", pessoa },
482 |         new Object[] { "DELETE FROM location_sharing WHERE idoso_id=?", pessoa },
```

A origem é conferida novamente. A transação troca dono, incrementa versões e retira acessos delegados e QR.

Experimente: O novo dono revisa vínculos e gera outro QR.

Limites: Proposta vale 24 horas. A troca técnica não transfere representação jurídica.

10. Contatos de emergência por pessoa

Na interface: Acompanhamento > Contatos e Idoso > Emergência.



Figura 9 — Contatos de emergência por pessoa

O que a funcionalidade faz: Pessoas diferentes podem ter contatos diferentes. Alterações são auditadas.

Fonte: `backend/src/main/java/com/cuidadosamente/backend/controlador/CuidadosController.java`, linha 2037. Os números à esquerda são as linhas reais.

```
2037 |   Map<String, Object> contatoEmergencia(String pessoa) {
2038 |     var contatos = banco.consultar(
2039 |       "SELECT nome,telefone,nome_alternativo AS " +
2040 |         "nomeAlternativo,telefone_alternativo AS telefoneAlternativo FROM " +
2041 |         "emergency_contacts WHERE idoso_id=?",
2042 |       pessoa
2043 |     );
2044 |     if (!contatos.isEmpty()) {
2045 |       return contatos.getFirst();
2046 |     }
2047 |     Map<String, Object> perfil = conversorJson.readValue(
2048 |       banco
2049 |         .consultar("SELECT perfil FROM users WHERE id=?", donoDosDados(pessoa))
2050 |         .getFirst()
2051 |         .get("perfil")
2052 |         .toString(),
2053 |       Map.class
2054 |     );
```

Busca contato pelo identificador da pessoa. Usa campos antigos do administrador somente como compatibilidade.

Experimente: Na emergência, escolha contato e confirme no discador.

Limites: Ligação depende de aparelho, rede e confirmação; não há acionamento automático.

11. Doses recorrentes

Na interface: Acompanhamento > Doses recorrentes.



Figura 10 — Doses recorrentes

O que a funcionalidade faz: Dois horários em dois dias geram quatro doses com estados próprios.

Fonte: `backend/src/main/java/com/cuidadosamente/backend/controlador/EvolucaoController.java`, linha 731. Os números à esquerda são as linhas reais.

```
731 |       for (LocalDate dia = dataInicial; !dia.isAfter(dataFinal); dia = dia.
plusDays(intervalo)) {
732 |         for (String hora : horarios) {
733 |           String registroId = UUID.randomUUID().toString();
734 |           doses++;
735 |           comandos.add(new Object[] {
736 |             "INSERT INTO registros(id,owner_id,idoso_id,tipo,titulo,data,horario,
detalhes,status,plano_id,fuso) VALUES(?,?,?,'medicamentos',?,?,?,?, 'pendente',?,?)",
737 |             registroId,
738 |             cuidados.donoDosDados(id),
739 |             id,
740 |             titulo,
741 |             dia.toString(),
742 |             hora,
743 |             detalhes,
744 |             planoId,
745 |             fuso,
746 |           });
747 |           comandos.add(
748 |             cuidados.evento(
```

O laço percorre dias e horários. Cada combinação recebe UUID, registro pendente e evento dose_programada.

Experimente: Cada dose fica pendente; cancelar remove só futuras pendentes.

Limites: Até um ano, oito horários e 730 doses. Intervalo inteiro de 1 a 30 dias. Fuso do plano é preservado.

12. Lembretes e privacidade

Na interface: Perfil > Configurações > Lembretes do cuidado.



Figura 11 — Lembretes e privacidade

O que a funcionalidade faz: O aviso genérico não revela o medicamento. Tocar não conclui o registro.

Fonte: `src/core/notificacoes/lembretes.native.ts`, linha 195. Os números à esquerda são as linhas reais.

```
195 |             body: preferencias.notification_details
196 |               ? r.titulo + ' - abra o aplicativo para conferir e registrar.'
197 |               : 'Há um cuidado programado. Abra o aplicativo para conferir.',
198 |             data: { conta: usuario.id, tipo: r.tipo, idosoId: r.idosoId },
199 |             sound: 'default',
200 |           },
201 |           trigger: {
202 |             type: Notifications.SchedulableTriggerInputTypes.DATE,
203 |             date: data,
204 |             channelId: CANAL,
205 |           },
206 |         });
207 |         // O logout pode ocorrer enquanto a chamada nativa ainda está agendando.
208 |         if (versao !== geracao) {
209 |           await Notifications.cancelScheduledNotificationAsync(identifier);
210 |           return 0;
```

A escolha muda texto detalhado ou genérico. O aviso leva conta e área; o gatilho DATE agenda o instante calculado.

Experimente: Abra diariamente e confira o registro ao tocar no aviso.

Limites: Até 50 avisos nas próximas 12 horas, antes da sessão expirar. Web não agenda; entrega precisa de validação física.

13. Offline e conflitos

Na interface: Perfil > Configurações > Sincronização offline.



Figura 12 — Offline e conflitos

O que a funcionalidade faz: Fila cifrada, até 100 alterações. O servidor revalida acesso e versão.

Fonte: `src/core/offline/armazenamento.native.ts`, linha 57. Os números à esquerda são as linhas reais.

```
057 |     const suporte = await db.getFirstAsync<Record<string, string>>('PRAGMA
cipher_version');
058 |     if (!suporte || !Object.values(suporte)[0]) {
059 |       throw new Error(
060 |         'Este aplicativo não tem SQLCipher. Instale uma build com armazenamento
protegido para usar offline.',
061 |       );
062 |     }
063 |     // A chave é hexadecimal gerada localmente; nenhum texto de usuário entra no
PRAGMA.
064 |     await db.execAsync('PRAGMA key = "x\'' + chave + '\'";');
065 |     await db.execAsync(
066 |       'CREATE TABLE IF NOT EXISTS cache (chave TEXT PRIMARY KEY, escopo TEXT NOT ' +
067 |         'NULL, dados TEXT NOT NULL, expires INTEGER NOT NULL); CREATE TABLE IF NOT '
+
068 |         'EXISTS pending (id TEXT PRIMARY KEY, escopo TEXT NOT NULL, caminho TEXT NOT
' +
069 |         'NULL, metodo TEXT NOT NULL, corpo TEXT NOT NULL, erro TEXT NOT NULL DEFAULT
' +
070 |         "'', instante INTEGER NOT NULL);",
071 |     );
072 |     banco = db;
073 |     return db;
074 |   } catch (erro) {
075 |     await db.closeAsync().catch(() => {});
```

O código exige SQLCipher. A chave aleatória fica no SecureStore e é aplicada antes das tabelas privadas.

Experimente: Reconecte. Em conflito, compare versões, copie o necessário e descarte a alteração local antes de editar o servidor.

Limites: Cache de até 5 milhões de caracteres, sem mídias. Revogação só é conhecida ao reconectar. Saída, troca de sessão ou recusa de acesso limpa cache e fila.

14. Recuperação e confirmação de e-mail

Na interface: Entrar > Recuperar acesso e Configurações.



Figura 13 — Recuperação e confirmação de e-mail

O que a funcionalidade faz: Códigos de uso único ficam como hash; recuperar remove todas as sessões da conta.

Fonte: `backend/src/main/java/com/cuidadosamente/backend/controlador/EvolucaoController.java`, linha 187. Os números à esquerda são as linhas reais.

```
187 |     var t = token(hash, "recuperacao");
188 |     String usuarioId = t.get("user_id").toString();
189 |     banco.executarTransacao(
190 |       new Object[] {
191 |         "UPDATE users SET senha=? WHERE id=?",
192 |         new BCryptPasswordEncoder(12).encode(senha),
193 |         usuarioId,
194 |       },
195 |       new Object[] { "DELETE FROM sessions WHERE user_id=?", usuarioId },
196 |       new Object[] { "DELETE FROM email_tokens WHERE user_id=? AND
kind='recuperacao'", usuarioId }
197 |     );
198 |   }
199 |
```

Valida tipo e prazo do código, grava nova senha BCrypt e remove sessões e códigos de recuperação.

Experimente: Entre novamente; para verificar e-mail, use o fluxo de confirmação em Configurações.

Limites: Validade de 30 minutos. Exige Resend, chave e remetente verificado. Sem configuração, a tela informa indisponibilidade.

15. Sessões e tentativas repetidas

Na interface: Perfil > Configurações > Sessões abertas.



Figura 14 — Sessões e tentativas repetidas

O que a funcionalidade faz: IDs de sessão permitem gerenciar acessos sem expor tokens utilizáveis.

Fonte: `backend/src/main/java/com/cuidadosamente/backend/controlador/EvolucaoController.java`, linha 104. Os números à esquerda são as linhas reais.

```
104 |   public void revogarOutras(
105 |     @RequestHeader(value = "Authorization", required = false) String autorizacao
106 |   ) {
107 |     banco.executar(
108 |       "DELETE FROM sessions WHERE user_id=? AND token<>?",
109 |       cuidados.usuarioDaSessao(autorizacao),
110 |       cuidados.gerarHash(autorizacao.substring(7))
111 |     );
112 |   }
113 |
```

A exclusão filtra a própria conta e preserva o hash da sessão atual. Outra conta não pode encerrar esses acessos.

Experimente: Se suspeitar de acesso indevido, recupere a senha.

Limites: Sessão dura 12 horas. Login: 10 tentativas por e-mail; autenticação: 40 POSTs por IP em 15 minutos. Proxy exige atenção operacional.

16. Captura pontual de GPS

Na interface: Idoso > Configurações > Compartilhar minha posição.



Figura 15 — Captura pontual de GPS

O que a funcionalidade faz: Só a conta idosa vinculada envia posição; a API guarda o ponto mais recente com precisão e horário.

Fonte: `backend/src/main/java/com/cuidadosamente/backend/controlador/EvolucaoController.java`, linha 1009. Os números à esquerda são as linhas reais.

```
1009 |     // A checagem e a gravação compartilham o bloqueio usado ao desligar
1010 |     // localização e revogar consentimento, evitando restaurar um ponto.
1011 |     synchronized (cuidados) {
1012 |       String usuarioId = cuidados.usuarioAutorizado(autorizacao);
1013 |       if (
1014 |         banco
1015 |           .consultar("SELECT 1 FROM elder_accounts WHERE idoso_id=? AND user_id=?",
id, usuarioId)
1016 |           .isEmpty()
1017 |       ) {
1018 |         throw new ResponseStatusException(
1019 |           HttpStatus.FORBIDDEN,
1020 |           "A posição deve vir do aparelho vinculado à pessoa idosa."
1021 |         );
1022 |       }
1023 |       cuidados.validarAcessoIdoso(id, usuarioId);
```

Este trecho exige o vínculo da conta idosa e revalida seu acesso. A continuação da função exige compartilhamento e confirmação explícita antes de gravar coordenadas.

Experimente: Toque em Enviar minha posição agora; para interromper, apague o ponto e desligue compartilhamento.

Limites: Sem rastreamento contínuo. Mais de 5 minutos é desatualizado; após 24 horas o ponto é retirado. Mapa externo é opcional.

17. Memórias e mídia privada

Na interface: Memórias > Adicionar e Arquivo privado.



Figura 16 — Memórias e mídia privada

O que a funcionalidade faz: Arquivos novos ficam fora do SQLite, identificados como media:UUID, sem URL pública.

Fonte: `backend/src/main/java/com/cuidadosamente/backend/controlador/MidiaController.java`, linha 87. Os números à esquerda são as linhas reais.

```
087 |   public ResponseEntity<FileSystemResource> download(
088 |     @RequestHeader(value = "Authorization", required = false) String autorizacao,
089 |     @PathVariable String id
090 |   ) {
091 |     var item = buscar(autorizacao, id, "ver_memorias");
092 |     Path arquivo = caminho(id);
093 |     if (!Files.isRegularFile(arquivo)) {
094 |       throw new ResponseStatusException(HttpStatus.NOT_FOUND, "Arquivo indisponível.
");
095 |     }
096 |     return ResponseEntity.ok()
097 |       .contentType(MediaType.parseMediaType(item.get("mime").toString()))
098 |       .header("Cache-Control", "no-store")
099 |       .header("X-Content-Type-Options", "nosniff")
100 |       .header("Content-Disposition", "attachment; filename=" + id)
101 |       .body(new FileSystemResource(arquivo));
```

buscar exige ver_memorias. Arquivo existente sai com no-store, nosniff e disposição de anexo.

Experimente: Para abrir, toque no arquivo; a API confere acesso novamente.

Limites: JPEG, PNG, WebP, MP3, WAV e MP4; 10 MB por arquivo, 200 MB por pessoa. Upload sem memória expira em 24 horas.

18. Exportação dos registros

Na interface: Acompanhamento > Exportar.



Figura 17 — Exportação dos registros

O que a funcionalidade faz: Inclui somente as áreas autorizadas, sem CPF, RG, endereço, GPS ou arquivos.

Fonte: `backend/src/main/java/com/cuidadosamente/backend/controlador/EvolucaoController.java`, linha 832. Os números à esquerda são as linhas reais.

```
832 |     for (String tipo : List.of("consultas", "medicamentos", "rotinas"))
833 |       if (cuidados.permissoesEfetivas(id, usuarioId).contains("ver_" + tipo)) {
834 |         registros.addAll(
835 |           banco.consultar(
836 |             "SELECT tipo,titulo,data,horario,detalhes,status FROM registros WHERE " +
837 |               "idoso_id=? AND tipo=? AND data BETWEEN ? AND ? ORDER BY data,horario",
838 |             id,
839 |             tipo,
840 |             inicio,
841 |             fim
842 |           )
843 |         );
844 |       }
845 |     return Map.of(
846 |       "nome",
847 |       banco.consultar("SELECT nome FROM idosos WHERE id=?", id).getFirst().
get("nome"),
848 |       "inicio",
849 |       inicio,
850 |       "fim",
```

Cada área exige ver_tipo. A consulta limita pessoa e período; a resposta identifica nome e data de geração.

Experimente: No nativo, gere o PDF e escolha destino de compartilhamento.

Limites: A cópia exportada fica fora do controle do aplicativo; escolha o destinatário com cuidado.

19. Leitura e navegação acessíveis

Na interface: Perfil > Configurações > Aparência e leitura.



Figura 18 — Leitura e navegação acessíveis

O que a funcionalidade faz: Fontes e altura de linha aumentam juntas. Botões informam estados e ícones decorativos deixam de disputar foco.

Fonte: `src/core/tema/paleta.ts`, linha 143. Os números à esquerda são as linhas reais.

```
143 |     if (typeof item === 'number' && (chave === 'fontSize' || chave === 'lineHeight'))
 {
144 |       resultado[chave] = item * escala;
145 |     } else if (
146 |       typeof item === 'string' &&
147 |       /Color$|^color$/.test(chave) &&
148 |       chave !== 'shadowColor'
149 |     ) {
150 |       resultado[chave] = adaptarCor(
151 |         item,
152 |         tema,
153 |         chave === 'backgroundColor' ? 'fundo' : chave === 'color' ? 'texto' :
'borda',
154 |       );
155 |     } else {
156 |       resultado[chave] =
157 |         item && typeof item === 'object' ? adaptarEstilos(item, tema, escala) : item;
158 |     }
159 |   }
160 |   return resultado as T;
161 | }
```

Multiplica fontSize e lineHeight pela escala, adapta cores por uso e percorre objetos internos.

Experimente: Use menu e abas autorizadas; valide também com leitor de tela.

Limites: Correções não substituem teste com pessoas idosas, TalkBack e VoiceOver. Veja ACESSIBILIDADE.md.

20. Consentimentos e direitos

Na interface: Configurações > Privacidade, consentimentos e meus direitos.



Figura 19 — Consentimentos e direitos

O que a funcionalidade faz: Revogar bloqueia cuidado, apaga posição GPS e invalida QR; a central continua acessível sem novo aceite.

Fonte: `backend/src/main/java/com/cuidadosamente/backend/controlador/EvolucaoController.java`, linha 961. Os números à esquerda são as linhas reais.

```
961 |       if (!permitido) {
962 |         comandos.add(new Object[] { "DELETE FROM location_points WHERE idoso_id=?",
id });
963 |         comandos.add(new Object[] { "DELETE FROM location_sharing WHERE idoso_id=?",
id });
964 |         comandos.add(new Object[] { "DELETE FROM qr_access WHERE idoso_id=?", id });
965 |       }
966 |       comandos.add(
967 |         cuidados.evento(
968 |           id,
969 |           usuarioId,
970 |           "acesso",
971 |           id,
972 |           permitido ? "consentimento_registrado" : "consentimento_revogado",
973 |           Map.of(),
974 |           Map.of("representacao", papel)
975 |         )
976 |       );
```

Revogação remove ponto, compartilhamento e QR na transação. A validação central bloqueia novas operações de cuidado.

Experimente: Indique autorização válida e digite AUTORIZAR ou REVOGAR para confirmar.

Limites: Declaração não prova representação. O contato real do controlador precisa ser definido. Providências e cópias dependem do processo administrativo.

21. Hospedagem, backup e métricas

Na interface: deploy/ e scripts/backup.py — ferramentas do mantenedor.



Figura 20 — Hospedagem, backup e métricas

O que a funcionalidade faz: Pacote AES-256-GCM reúne SQLite e mídia, manifesto e hashes. Métricas têm só contagens e duração.

Fonte: `scripts/backup.py`, linha 117. Os números à esquerda são as linhas reais.

```
117 |     if destino.exists():
118 |         raise ValueError("A restauração exige uma pasta nova; não sobrescreve dados.
")
119 |     destino.parent.mkdir(parents=True, exist_ok=True)
120 |     # Autenticidade, caminhos e checksums são validados antes de publicar a
restauração.
121 |     with tempfile.TemporaryDirectory(prefix="cam-restore-", dir=destino.parent) as
temp:
122 |         staging = Path(temp)
123 |         pacote = staging / "backup.zip"
124 |         decifrar(args.archive.resolve(), pacote, chave(args.key_file))
125 |         prepared = staging / "restaurado"
126 |         prepared.mkdir(mode=0o700)
127 |         with zipfile.ZipFile(pacote) as z:
128 |             manifest = json.loads(z.read("manifest.json"))
129 |             if manifest.get("formato") != 1:
130 |                 raise ValueError("Versão de backup inválida.")
131 |             esperado = manifest["arquivos"]
```

Recusa sobrescrever pastas. A cópia é autenticada em staging e só é publicada depois das verificações.

Experimente: Restaure em pasta nova e valide antes de planejar troca do ambiente.

Limites: Infraestrutura e contas externas não foram ativadas. Docker, systemd e nuvem requerem teste no servidor; restauração local usa dados sintéticos.

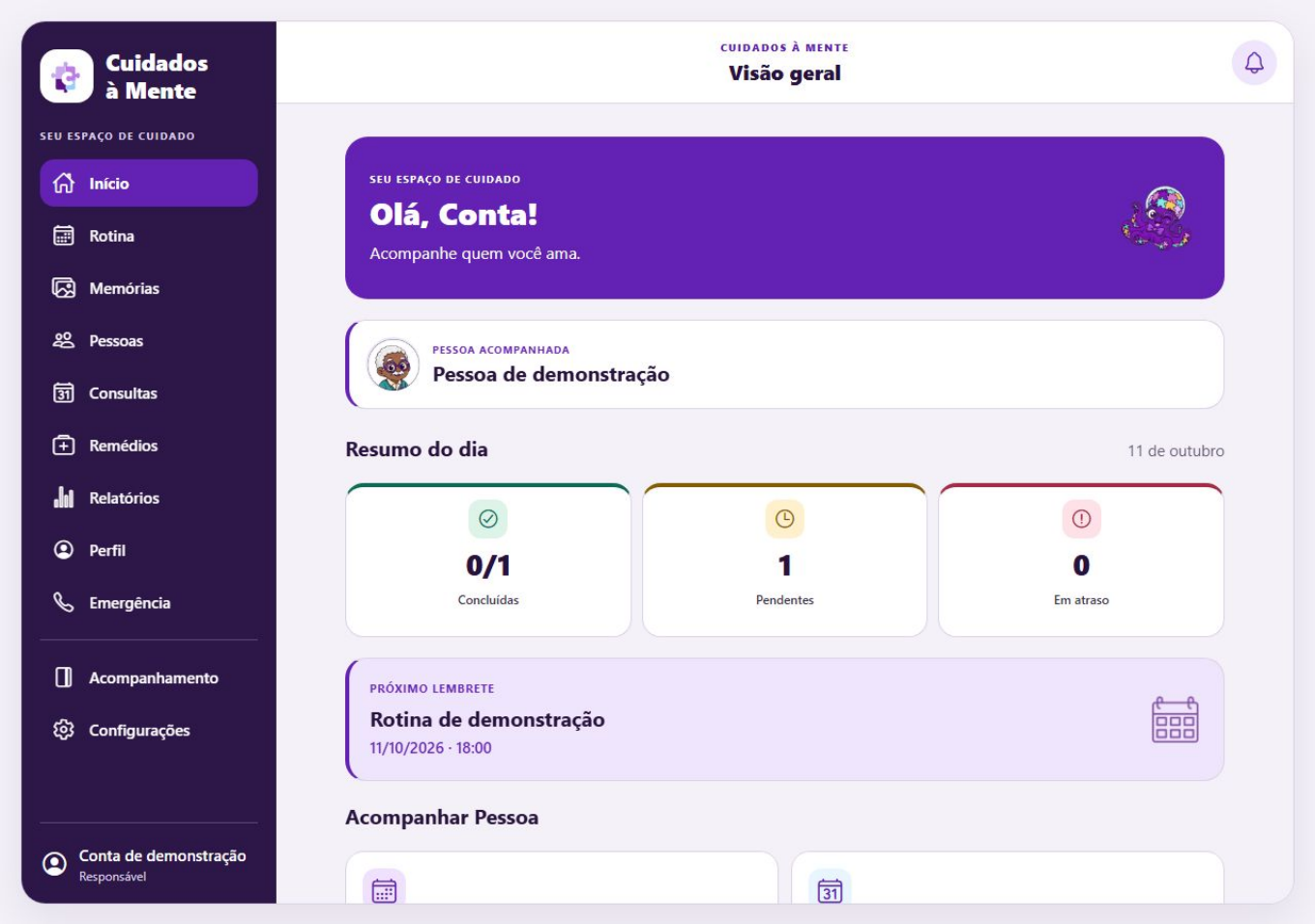
22. Como testar o que aprendeu

1. Leia cliente-api e contexto-autenticacao para sessão e falhas.
2. Leia CuidadosController para permissão e versão dos registros.
3. Leia EvolucaoController e MidiaController para as funções novas.
4. Leia BancoDados para migrações e transações.
5. Confira ALTERACOES_E_TESTES.md e use dados temporários.

Verificação	O que avalia
npm run typecheck	Tipos TypeScript
npm run lint	Regras estáticas de React
npm run format:check	Padrão de formatação dos fontes
npm run test:api-client	Sessão, HTTP e recibo de criação
npm run test:offline	Concorrência, troca de conta e fila cifrada
npm run test:notifications	Agenda, conta e pessoa do lembrete
npm run test:core	Fuso, fonte e contraste
Maven package	Java, contexto e regras no SQLite
npm run test:integration	HTTP, QR, CORS e permissões
test_backup.py	Restauração, adulteração e sobrescrita
Avaliação nativa e com pessoas	Entrega de avisos, GPS e usabilidade reais

A cópia imediatamente anterior ao clean code está em work/clean-code/antes-clean-code.zip. A versão das recomendações continua preservada em work/recomendacoes-original. A tabela de recomendações distingue código pronto, configuração externa e avaliação com usuários.

23. Identidade visual e navegação adaptável



Navegação lateral e conteúdo centralizado

Fonte: `src/core/tema/paleta.ts`, linha 1. Os números à esquerda são as linhas reais.

```
001 | export const colors = {
002 |   purple: '#6422B5',
003 |   ink: '#28133F',
004 |   lavender: '#B784F2',
005 |   pale: '#EFE5FC',
006 |   nav: '#F5F0FB',
007 |   surface: '#F4F1F8',
008 |   line: '#DCD2E6',
009 |   muted: '#665B73',
010 |   body: '#45374F',
011 |   green: '#DEF4E9',
012 |   yellow: '#FFF0CE',
013 |   red: '#FFE2E7',
014 |   sidebar: '#2D1748',
```

Estes tokens dão a mesma identidade ao cabeçalho, aos botões e aos cartões. Roxo destaca ações; verde, âmbar e vermelho identificam situações do cuidado, acompanhadas de texto.

Fonte: `src/components/layout/estrutura-aplicativo.tsx`, linha 63. Os números à esquerda são as linhas reais.

```
063 |   const desktop = width >= 1024;
```

A largura da janela escolhe o menu lateral a partir de 1024 pixels. Em telas menores, o aplicativo mostra uma barra inferior e o botão Mais.

Fonte: `src/components/layout/estrutura-aplicativo.tsx`, linha 111. Os números à esquerda são as linhas reais.

```
111 |   const leituraAmpliada = (usuario?.preferencias?.font_scale || 1) * fontScale >= 1.
4;
112 |   const prioridade: SecaoAplicativo[] = elder
113 |     ? leituraAmpliada
114 |       ? ['inicio', 'emergencia', 'perfil']
115 |       : ['inicio', 'rotinas', 'emergencia', 'perfil']
116 |     : leituraAmpliada
117 |       ? ['inicio', 'rotinas', 'perfil']
118 |       : ['inicio', 'rotinas', 'memorias', 'perfil'];
```

O tamanho escolhido na conta é combinado com a escala de fonte do aparelho. Com letras ampliadas, uma opção passa para Mais, deixando espaço para ler os rótulos. As permissões continuam filtrando os destinos.

Experimente: abra o painel em uma janela estreita e depois larga. Em Configurações, salve letras de 160% e confira a barra inferior.

24. Login do idoso preparado pelo proprietário

Fonte: `backend/src/main/java/com/cuidadosamente/backend/controlador/CuidadosController.java`, linha 397. Os números à esquerda são as linhas reais.

```
397 |     if ("idoso".equals(linhas.getFirst().get("tipo"))) {
398 |       if (((Number) linhas.getFirst().get("password_login_enabled")).intValue() != 1)
 {
399 |         throw erroDadosInvalidos(
400 |           "Use o QR Code ou peça ao responsável para configurar seu e-mail e senha."
401 |         );
402 |       }
403 |       String usuarioId = linhas.getFirst().get("id").toString();
404 |       var vinculo = banco.consultar(
405 |         "SELECT idoso_id FROM elder_accounts WHERE user_id=?",
406 |         usuarioId
407 |       );
408 |       if (vinculo.isEmpty()) {
409 |         throw new ResponseStatusException(HttpStatus.FORBIDDEN, "Conta sem vínculo.")
;
410 |       }
411 |       validarAcessoIdoso(vinculo.getFirst().get("idoso_id").toString(), usuarioId);
412 |     }
413 |     banco.executar("DELETE FROM auth_limits WHERE key=?", "login:" + gerarHash(email)
);
414 |     return criarSessao(linhas.getFirst().get("id").toString());
415 |   }
416 |
```

A API só permite senha quando este acesso foi configurado. Depois exige vínculo com uma pessoa e verifica se a autorização continua válida. O tipo da conta e suas permissões permanecem os mesmos; o aceite dos documentos não é registrado pelo login.

A rota PUT /idosos/ID/acesso-senha é exclusiva do proprietário. Ela valida credenciais, guarda o hash BCrypt, invalida sessões e códigos anteriores e registra um evento sem senha. A alternativa QR continua disponível.

25. Como ler o código após o clean code

As funcionalidades, rotas, permissões e dados continuam iguais. A refatoração separou declarações, explicitou blocos e substituiu abreviações por nomes como estilosDaTela, ocupado e usuarioId.

BancoDados inicia em quatro etapas: criarTabelasIniciais, migrarPessoasERegistros, migrarSessoesEVinculos e prepararContinuidadeEPrivacidade. Elas executam as mesmas instruções na mesma ordem. O HTML de impressão agora fica em src/features/cuidados/relatorios/relatorio-cuidados.ts, separado da obtenção e do compartilhamento na tela.

Os limites do cache e da agenda receberam nomes, e identificadorDoLembrete reúne a construção do mesmo identificador antes repetida. Leia CLEAN_CODE_2026-10-12.md e GUIA-CODIGO.md para manter o padrão e conferir os testes.